

Deep Learning | Final Project | PyTorch

Noa Levy & Shahar Nuss

[GITHUB repository link](#)

Image Classification with CNNs in PyTorch – Experimental Study

Task: Binary classification – Real vs. AI-generated portrait images
Dataset: 200 images (100 real, 100 AI), derived from Exercise 1
Goal: Compare strategies: regularization, augmentation, and transfer learning – focusing on convergence, stability, and generalization (not only accuracy)

Approach: From-scratch CNNs → regularization → CIFAR pretraining → ResNet50 (ImageNet) transfer learning

Key Findings

- BatchNorm and ImageNet transfer learning (ResNet50) provided the largest gains in convergence speed and stability.
- Moderate regularization (dropout=0.2, wd=1e-4) improved validation stability, while final accuracy saturated due to the small dataset.
- Strong augmentation increased difficulty and did not improve final test accuracy in this setting.
- Final model: ResNet50 (ImageNet) – Test Acc: 100%

Problem & Dataset

Definition

Classification – Real vs AI-generated portrait

Dataset: Exercise 1, N=200 (100 Real / 100 AI) Split (seeded):
70/15/15 (Train=140, Val=30, Test=30), seed=42 Input:

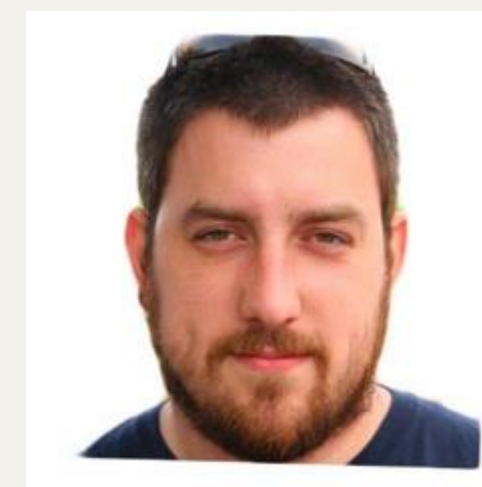
Resize to 224×224 Normalization:

From-scratch CNNs: dataset mean/std Transfer
learning (ResNet50): ImageNet mean/std

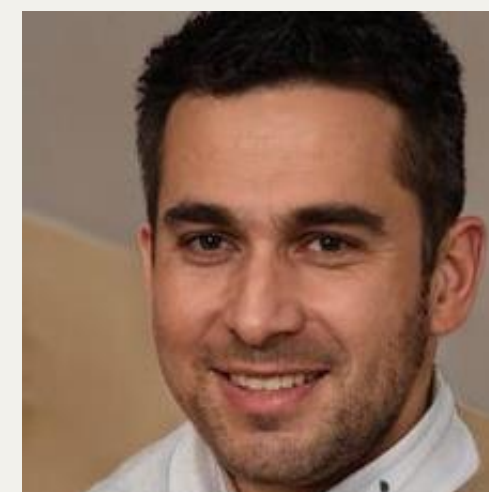
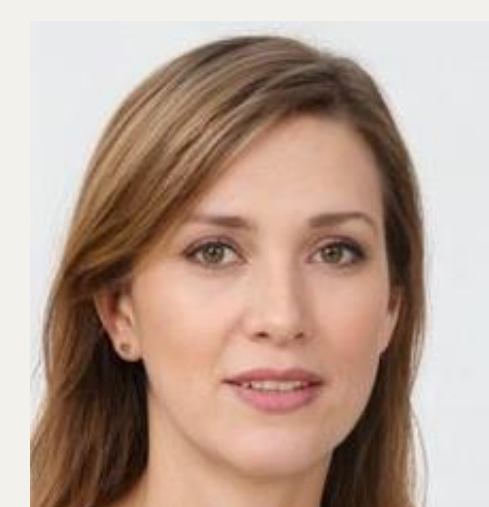
Metrics: Accuracy + cross-entropy loss (reported on Train/Val;
final Test per step) Step: We report Train/Val curves + final Test
metrics per step

**Goal: analyze stability, convergence speed,
and generalization – not only accuracy.**

AI



Real



Methodology and Experimental Protocol

Fixed seed + split

Same metrics (acc + CE loss) Curves (train/val) + test metrics per step Fair comparisons (what is held constant) What changes per step (BN / reg / aug / transfer)

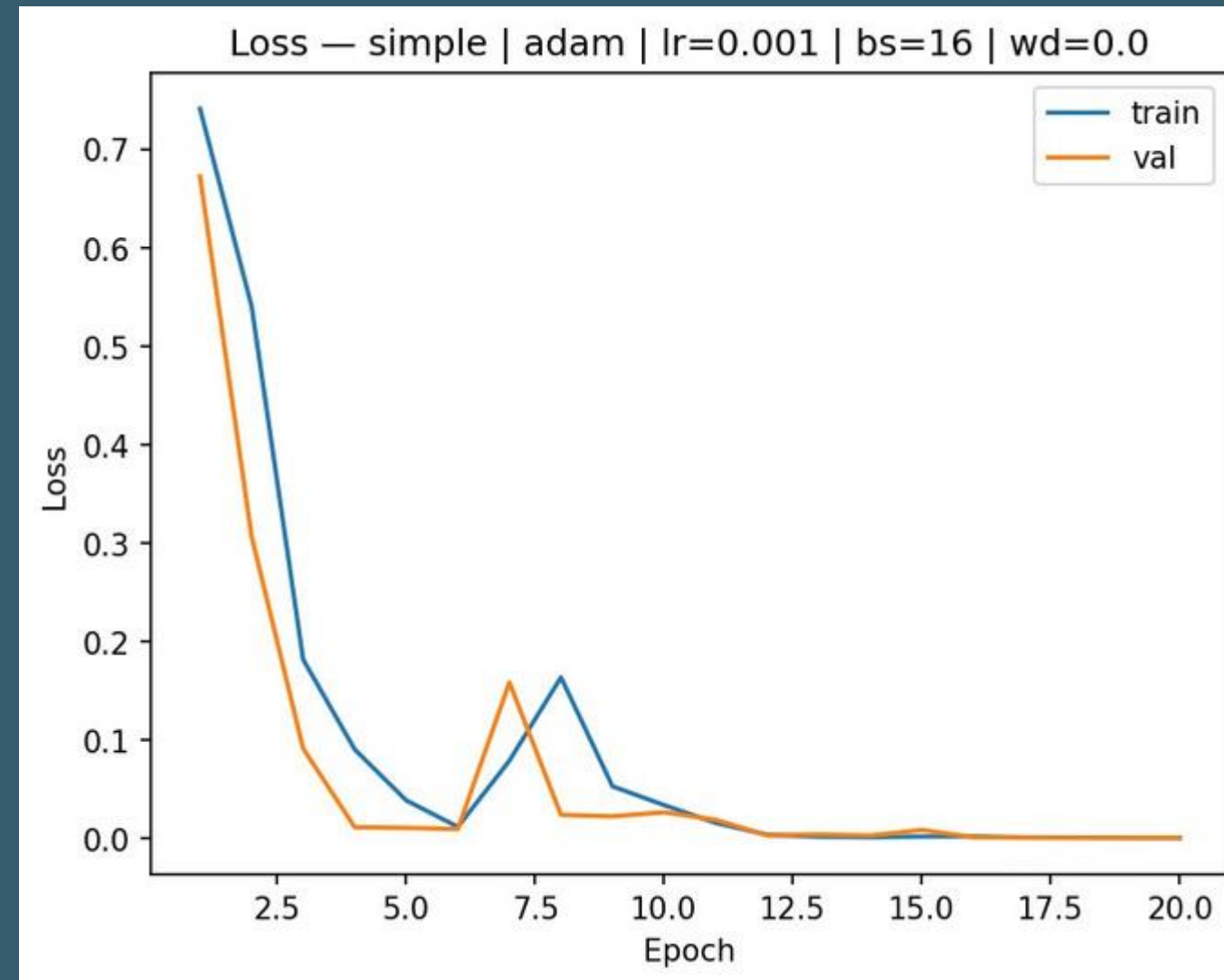
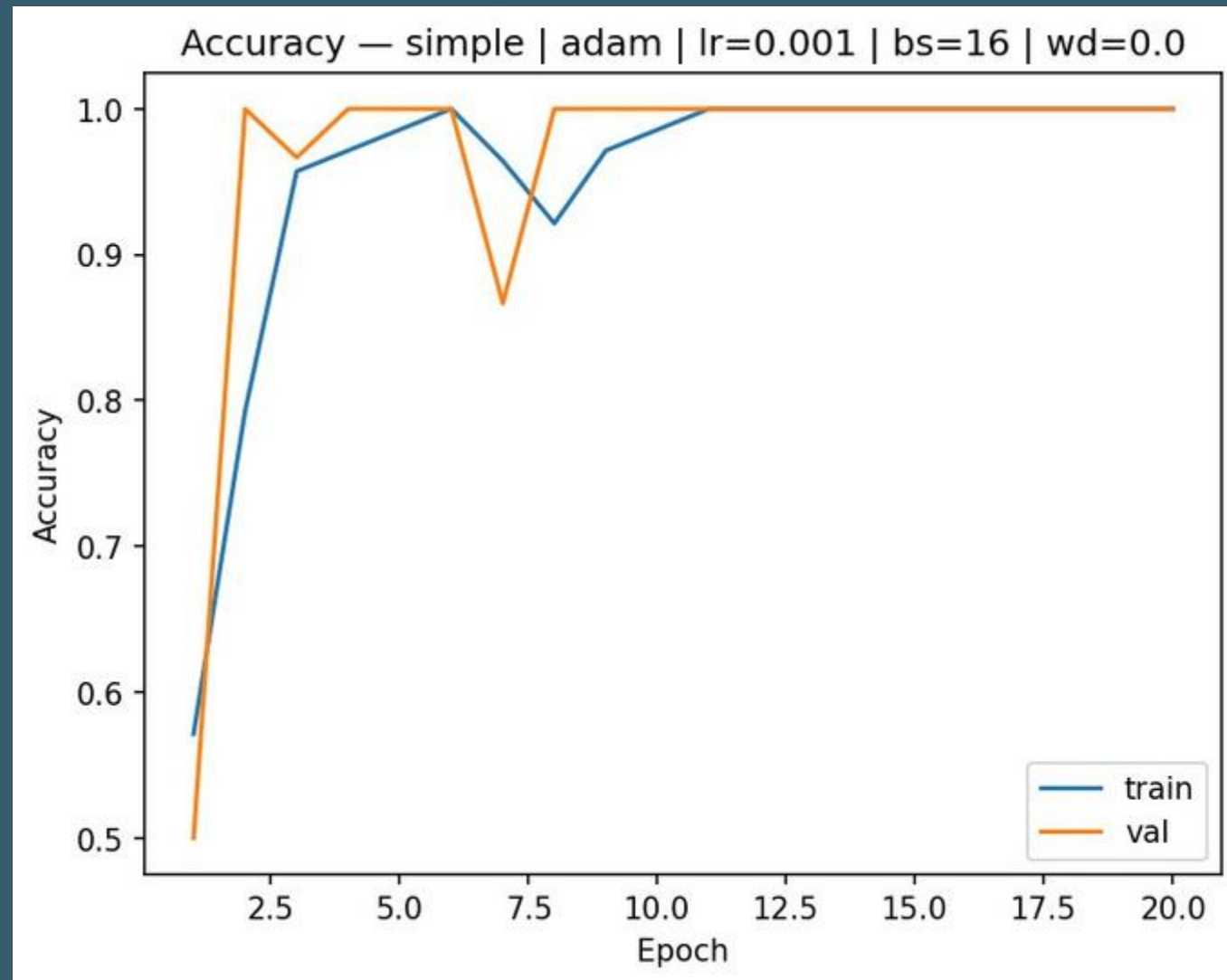
Focus: Special emphasis will be placed on analyzing the stability and optimization behavior of the models, beyond merely achieving peak accuracy.

Step 1A- Baseline (SimpleCNN + Adam)

Model: SimpleCNN (shallow conv blocks → pooling → FC)

Setup: Adam, lr= 1e-3 (seed=42, split 70/15/15)

Test: Acc=93.3%, Loss=0.29



Train/Val curves (seed=42, split 70/15/15).

Fast convergence with minor spikes (small val set,

N=30) shows occasional instability/spikes due to small val set (N=30), but overall convergence is [green](#)

[coded](#)

Step 1B- Baseline Architecture Comparison (SimpleCNN vs DeeperCNN)

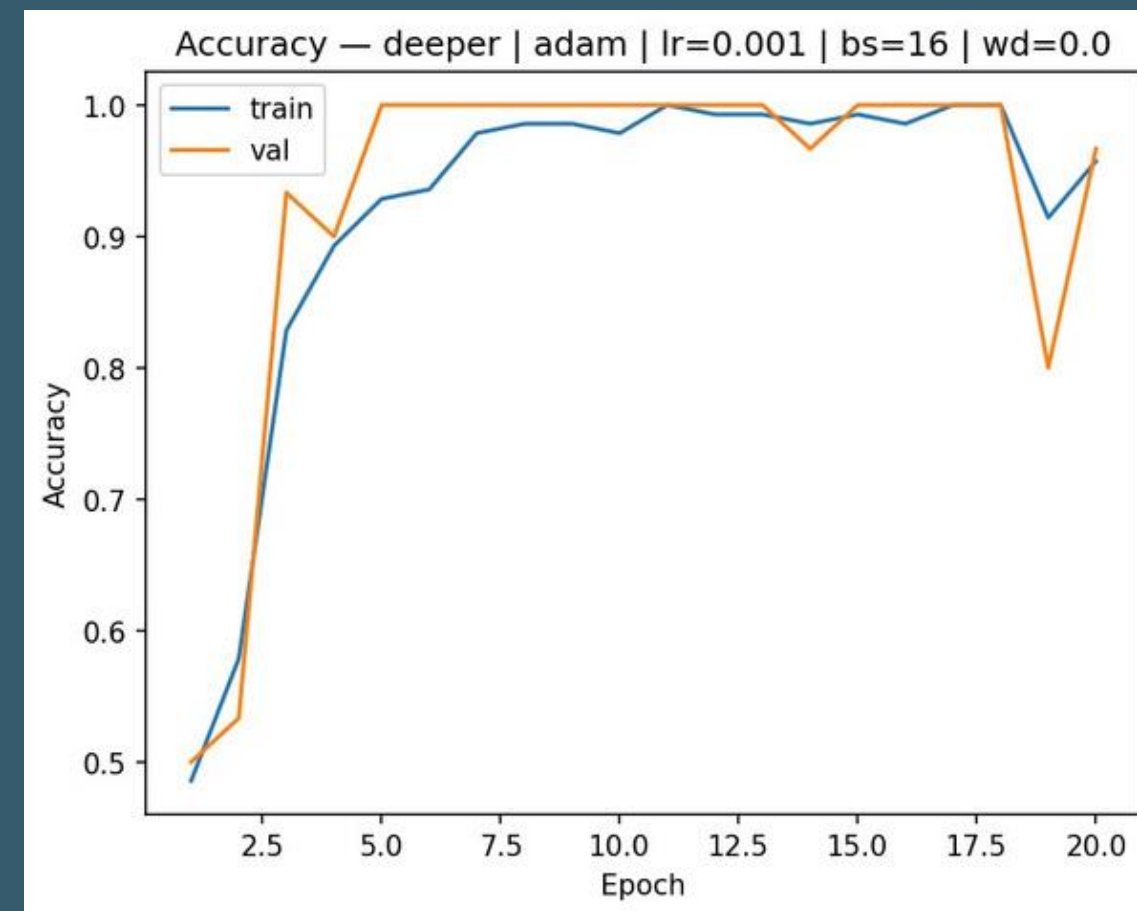
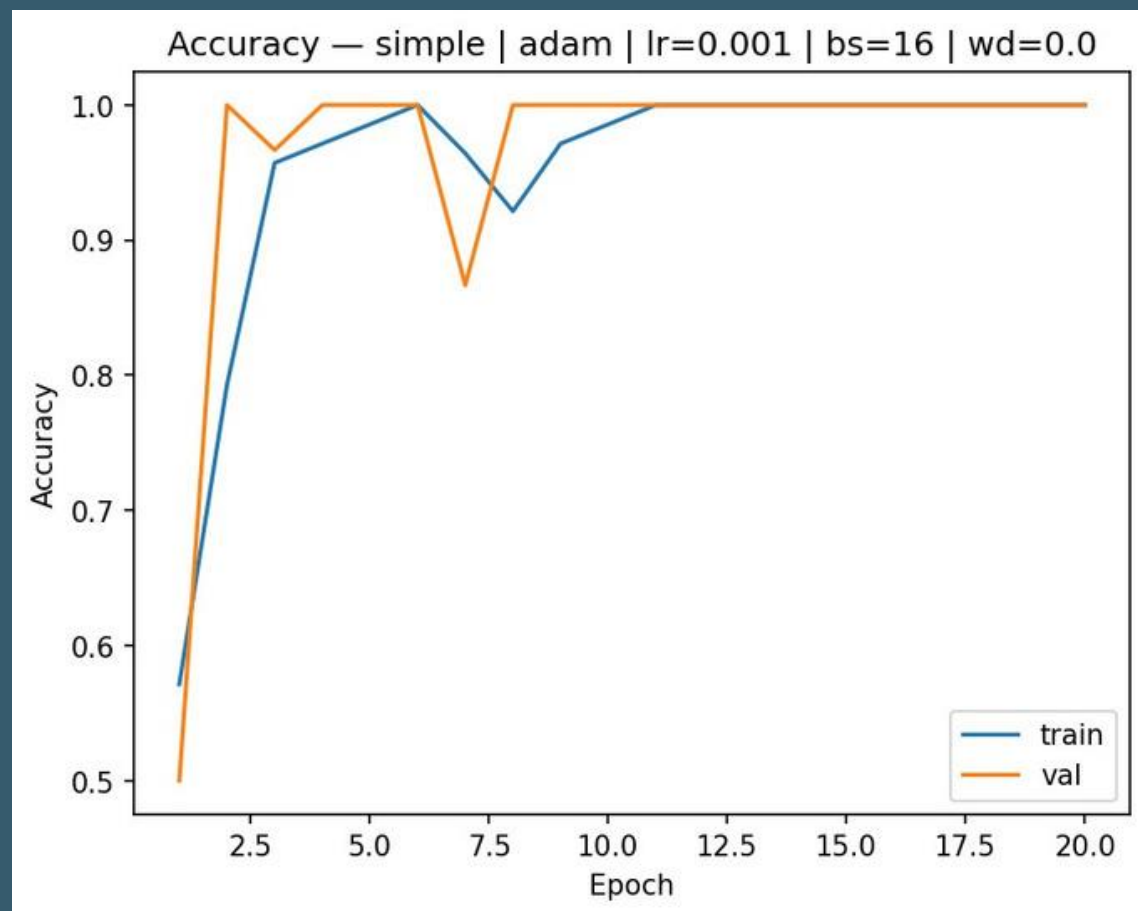
Goal: Compare two CNN architectures trained from scratch with the same optimizer to assess the effect of depth/capacity.

Setup (same for both): Adam, lr=1e-3, bs=16, seed=42, split 70/15/15, epochs=20-30, wd=0.0

Results (fill numbers):

- SimpleCNN + Adam: Test Acc = 93.3%, Test Loss = 0.29
- DeeperCNN + Adam: Test Acc = 96.6%, Test Loss = 0.062
- Observation (1-2 bullets):
 - DeeperCNN reaches high validation accuracy faster and stays near saturation for most epochs.
 - Minor validation dips appear occasionally due to the small validation set (N=30), but both models generalize well.

Conclusion: Deeper CNN provides higher capacity and achieves better (or similar) generalization than the baseline, so it is a stronger architecture for follow-up experiments.



Step 1C- Optimizer Comparison (Adam vs SGD on DeeperCNN)

Goal: Compare optimization behavior on the same architecture to evaluate convergence speed and stability.

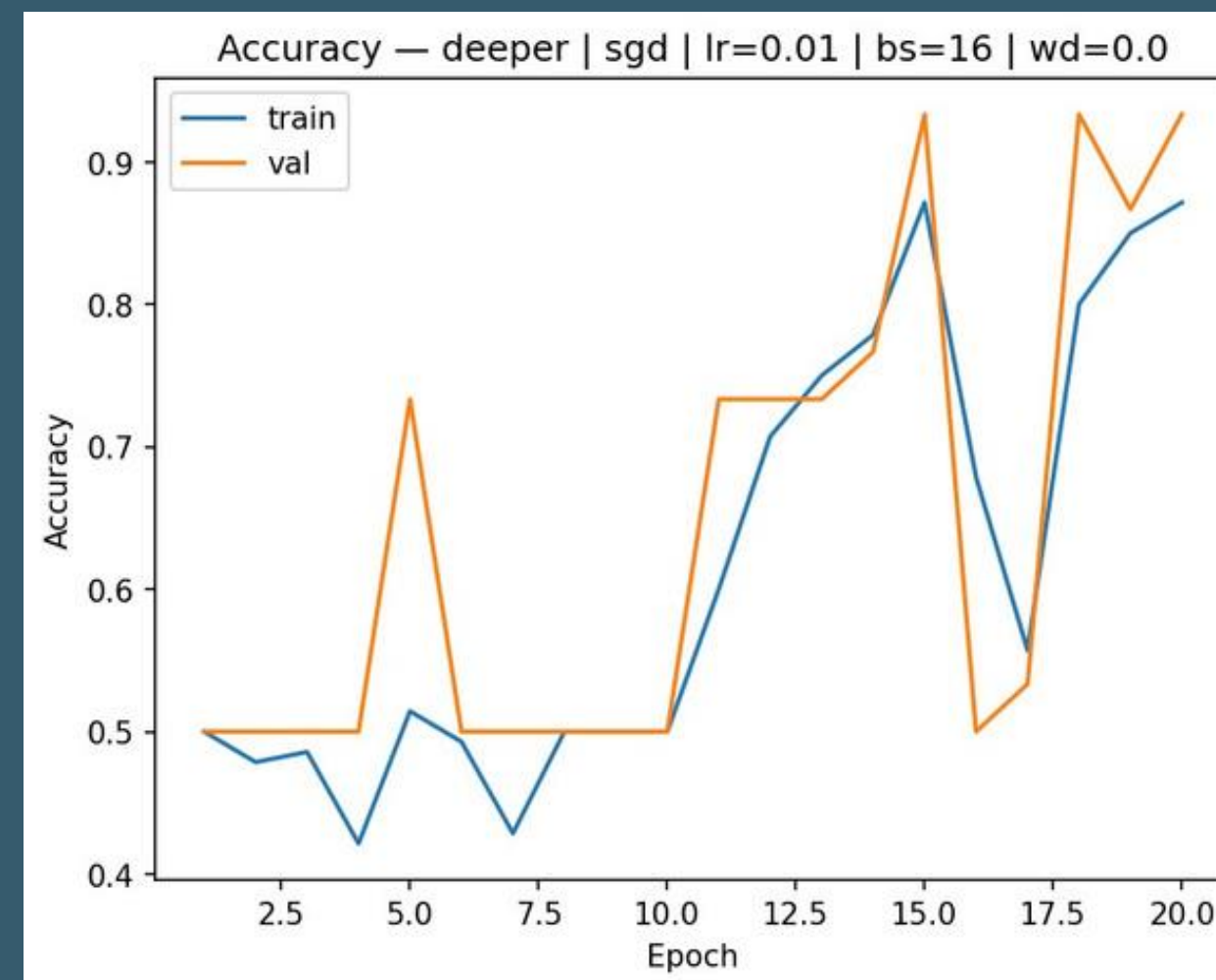
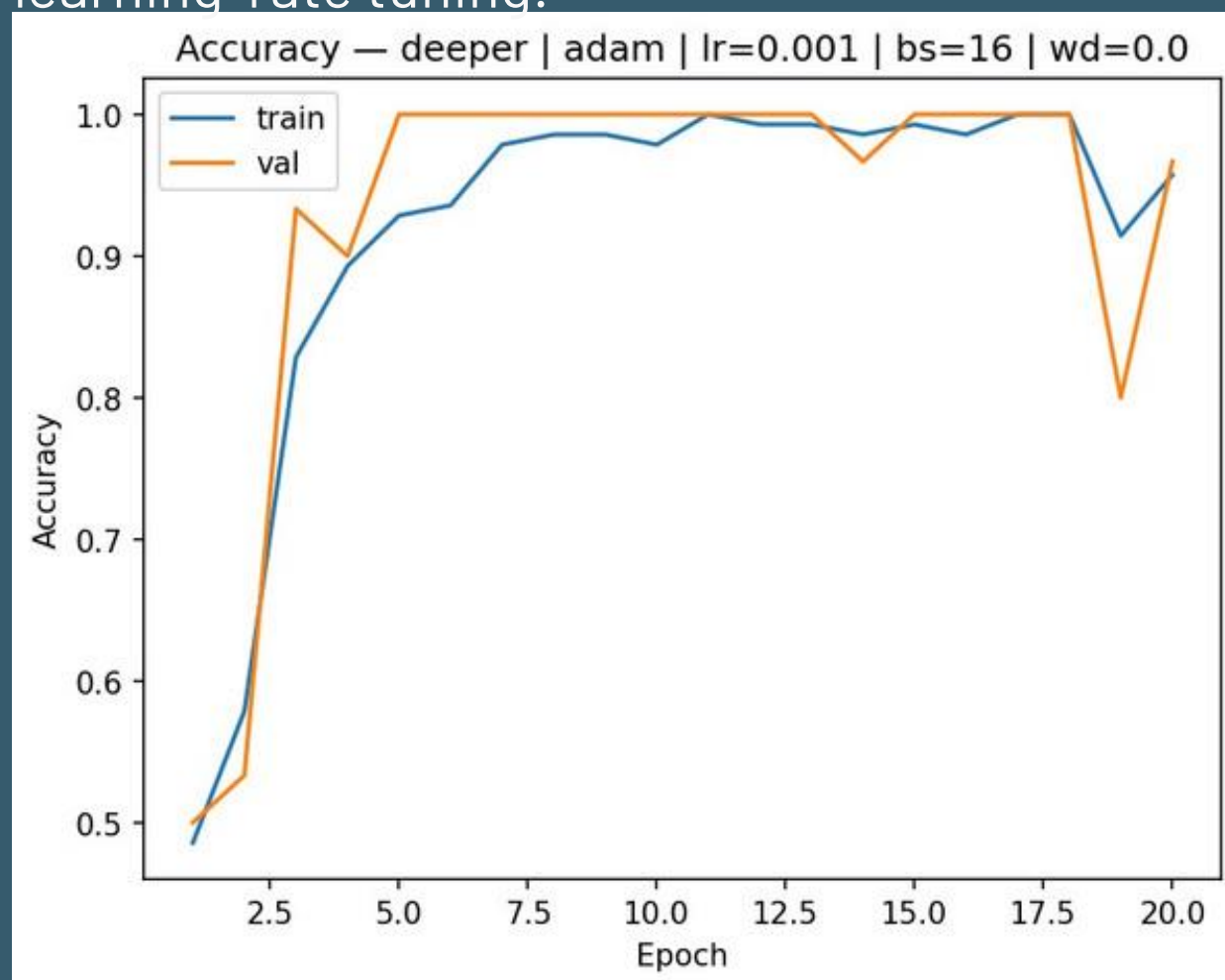
- Setup (same architecture): Model: DeeperCNN Split: 70/15/15 (seed=42, val set N=30) Batch size: 16, Weight decay: 0.0
 - Runs: Adam (lr = 1e-3) SGD + momentum (lr = 0.01, momentum = 0.9)

Observations:

- Adam converges rapidly to very high train/val accuracy and remains mostly stable.
- SGD progresses more slowly and shows higher variance in validation accuracy, indicating stronger sensitivity to hyperparameters.
- Validation spikes are amplified by the small validation set (N=30).

Conclusion:

Adam is the most stable and efficient optimizer for training from scratch in this project. SGD can work, but typically requires careful learning-rate tuning.



Optimizer comparison is conducted on the same architecture (DeeperCNN) for a fair comparison

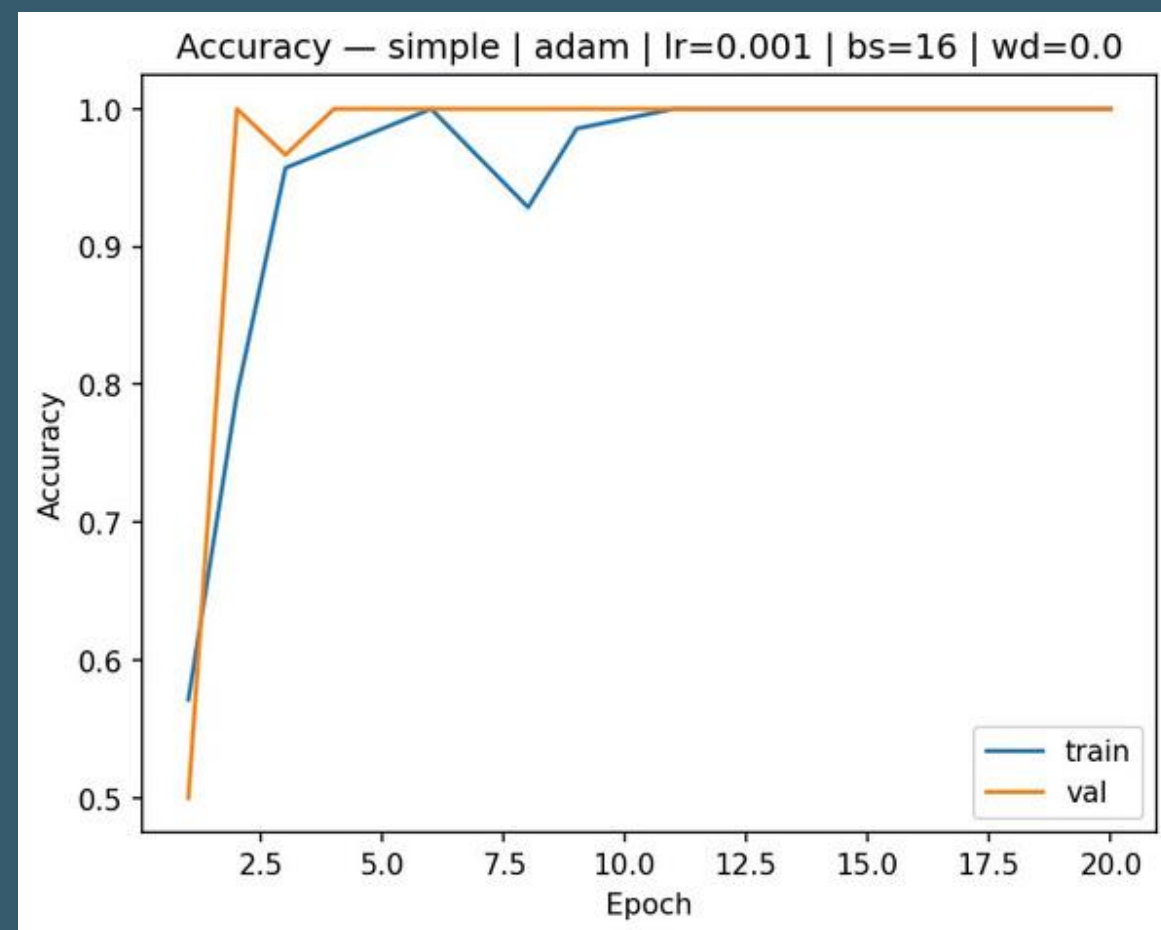
Step 2- Optimization- BatchNorm and Dropout

Change: Added Batch Norm (Dropout=0.2 kept)

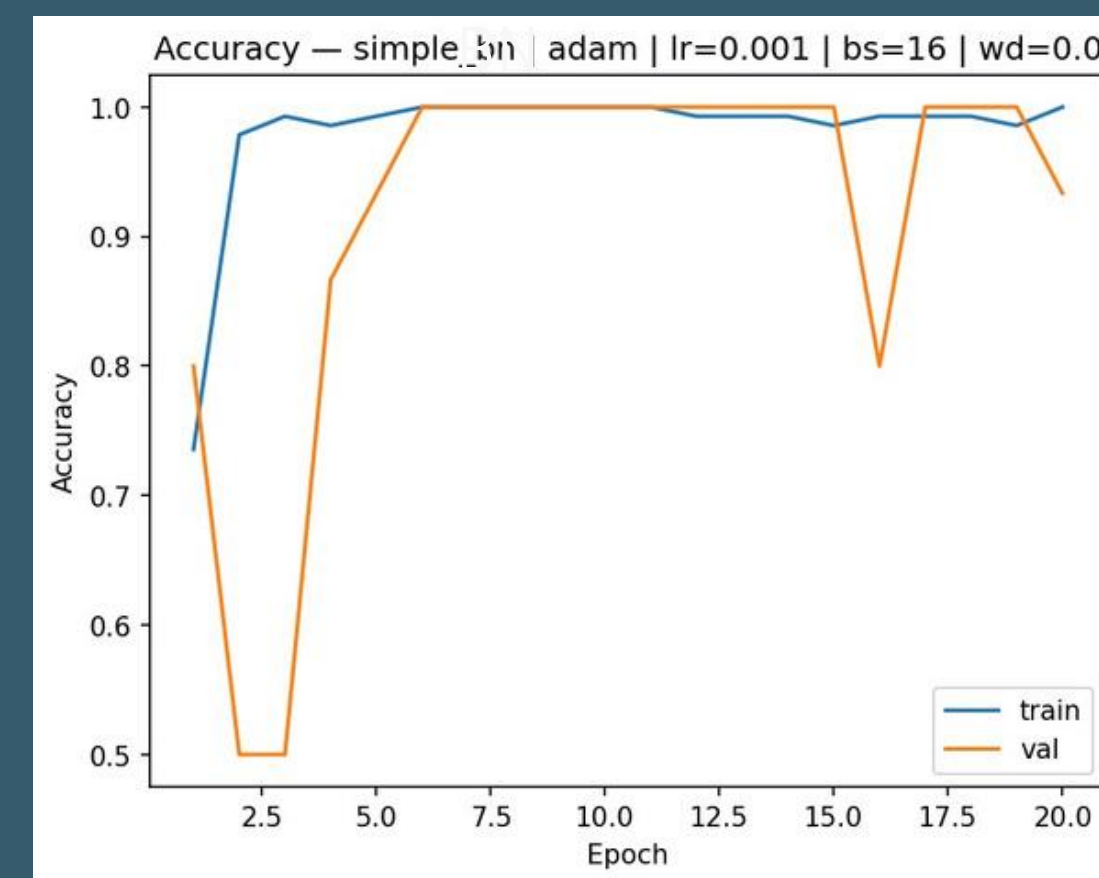
Effect: Faster convergence; Validation Accuracy: ~95-100% (spiky due to N=30) Test (BN model): Accuracy = 100%, Loss ≈ 0 BN speeds up convergence and improves optimization stability; however, with a small validation split (N=30), validation accuracy can show spikes. Dropout mainly improves training stability.

Occasional validation drops are expected with N=30; overall convergence is faster and more stable than without BN

NO BN



WITH



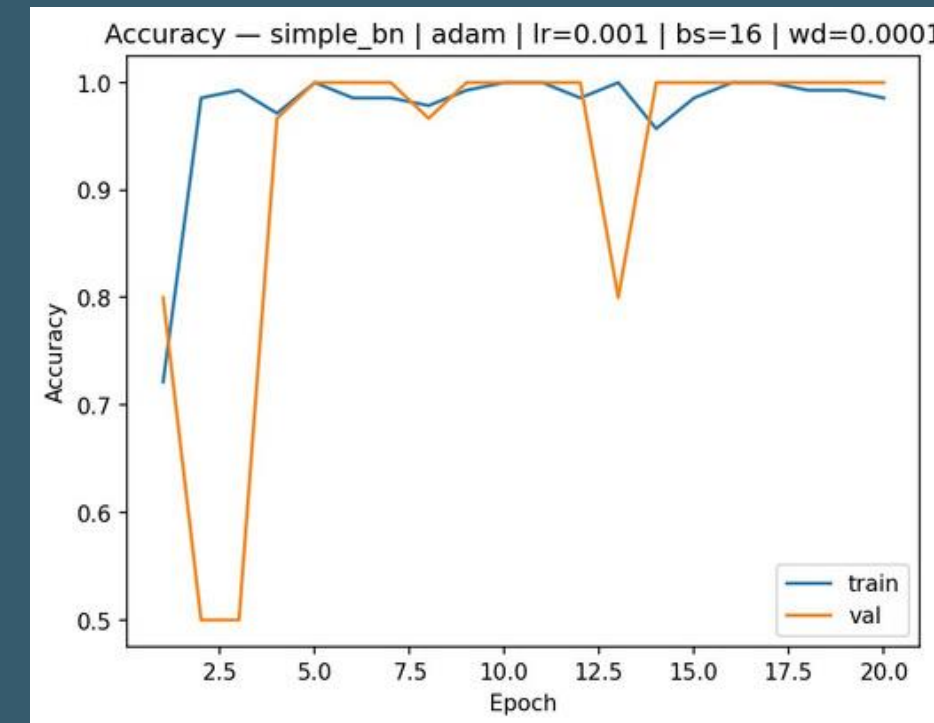
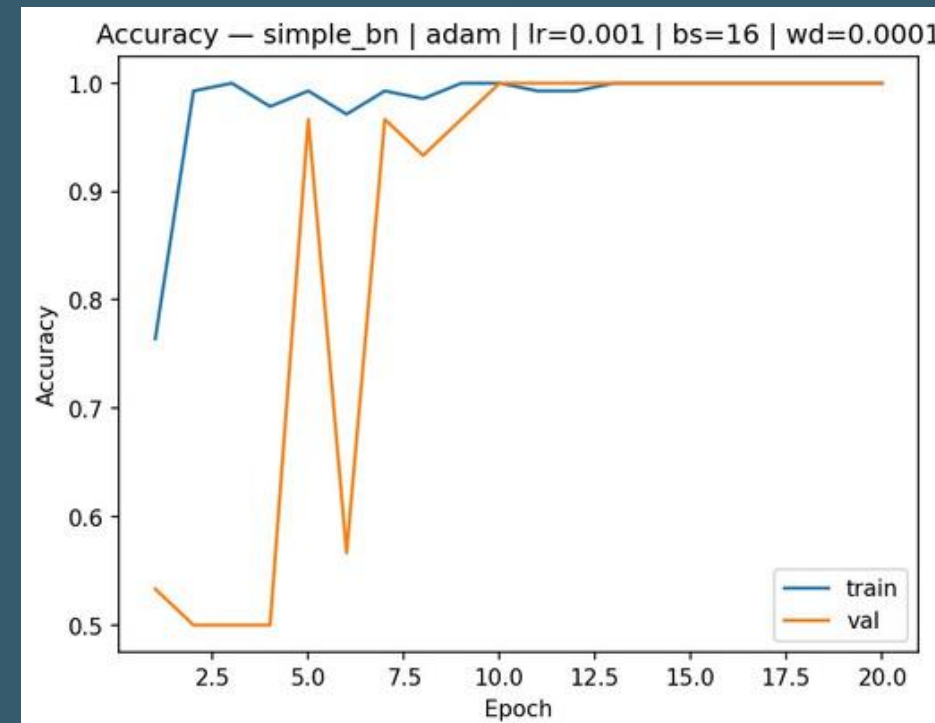
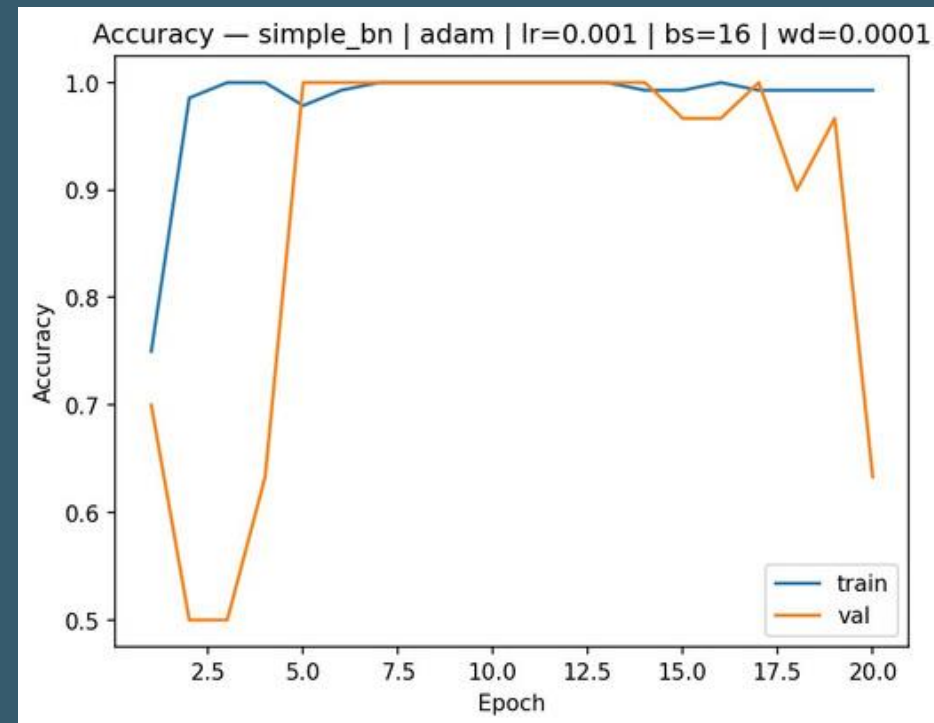
Step 3- Regularization

Regularization improves stability (not final accuracy)

Ablations
Dropout 0.0

Dropout 0.2

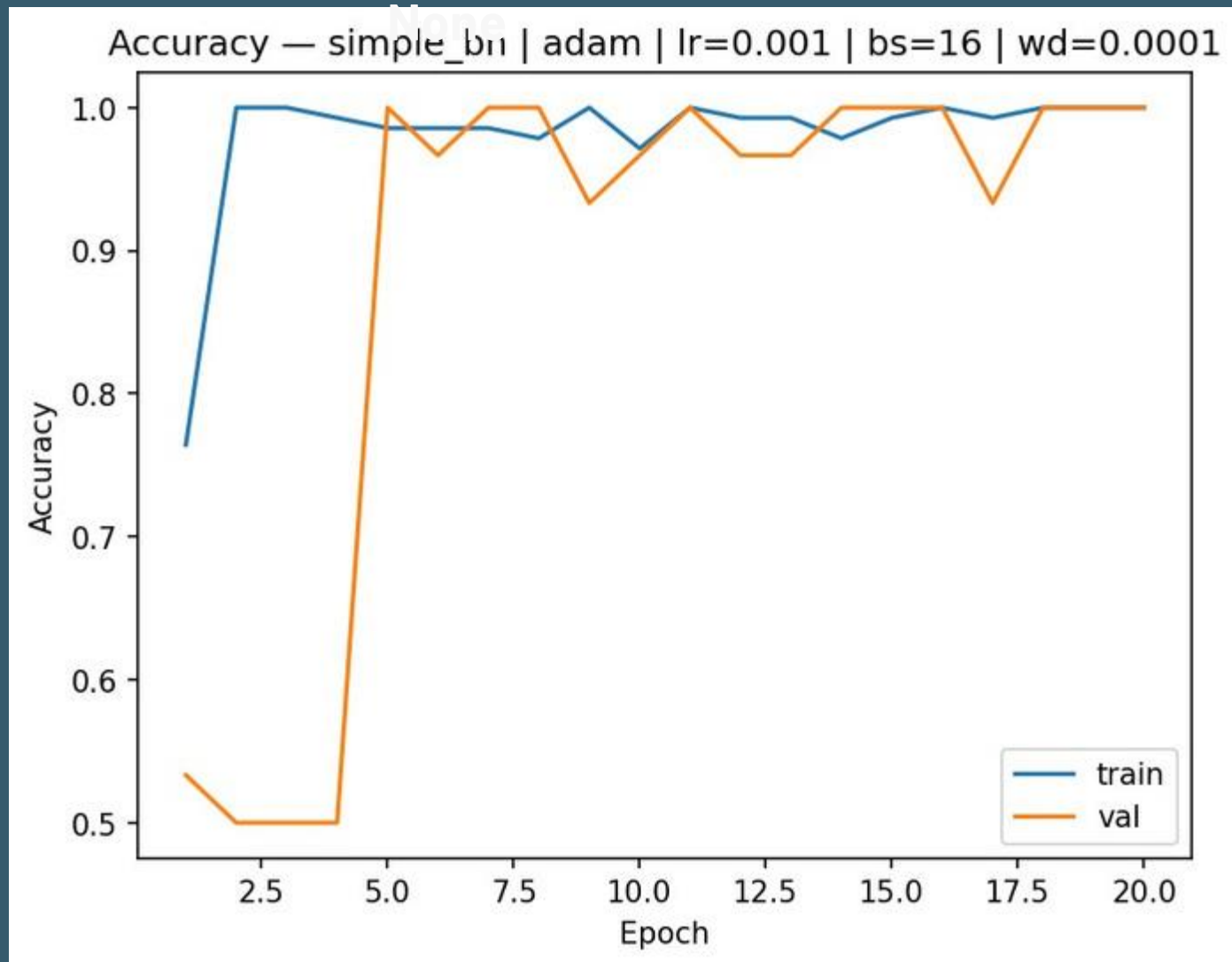
Dropout 0.5



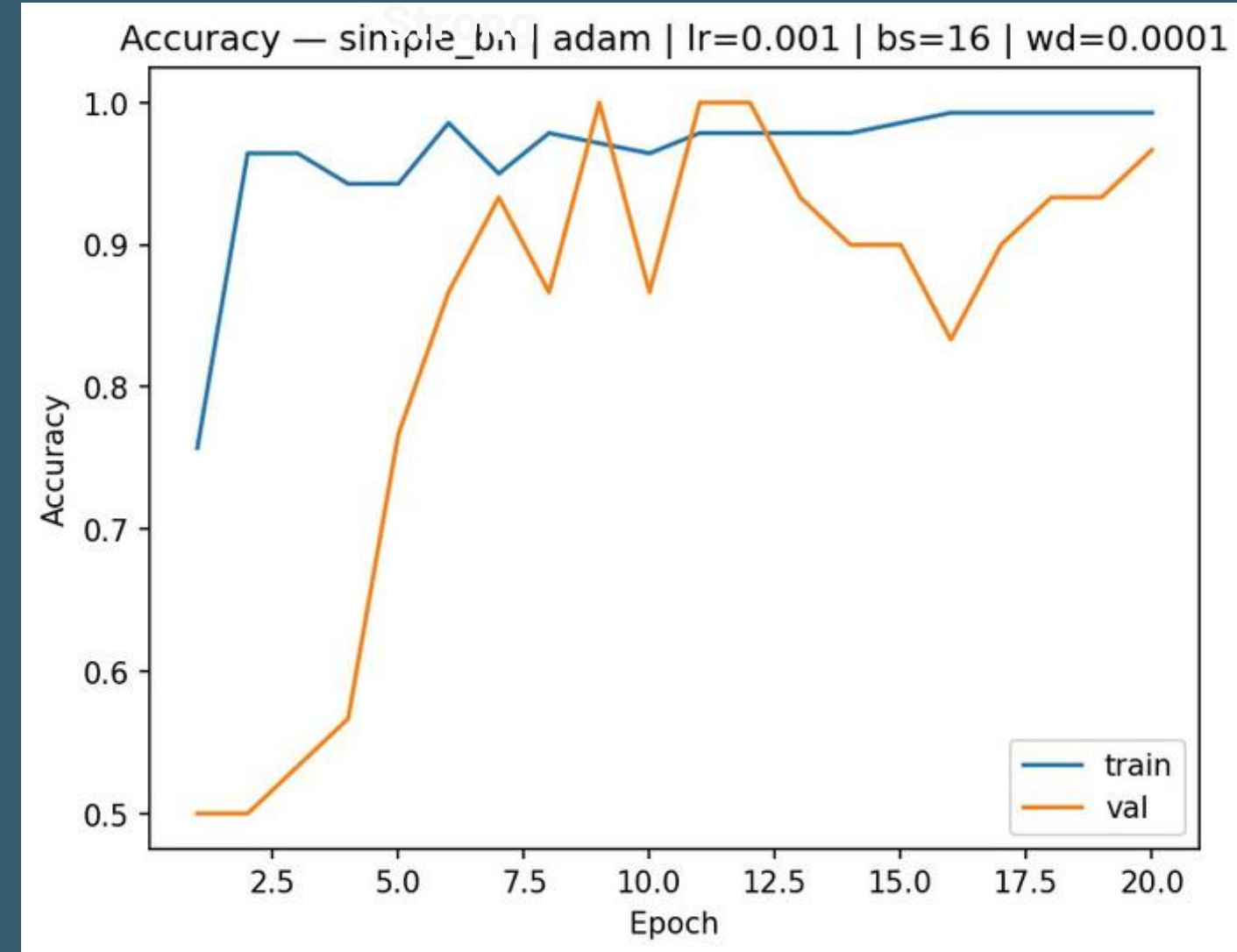
- Dropout (0.0 / 0.2 / 0.5): all reached 100% test acc
- Stability: p=0.0 showed validation instability/fluctuations; p=0.2 was most stable; p=0.5 converged slower
- Weight Decay (0 / $1e-4$ / $1e-3$): all reached 100% test acc
- Best stability: wd= $1e-4$ reduced validation spikes without slowing convergence
- Takeaway: With a small/easy dataset, regularization mainly reduces variance in validation curves rather than improving final test accuracy.

Step 3- Strong Data Augmentation

Augmentation:



Augmentation:



- Goal: encourage robustness via stronger invariances
 - Training effect: noisier curves + slower convergence
 - Test: Acc≈96.7% (augmentation did not improve peak performance here)
- *Augmentation increases invariances but makes optimization harder; for small/easy dataset it can reduce peak val.

Steps 4–5 – Pretraining on CIFAR and Fine-

Step 4 (Pretrain): CIFAR-10 accuracy improved $\sim 28\% \rightarrow \sim 56\%$ (feature learning)

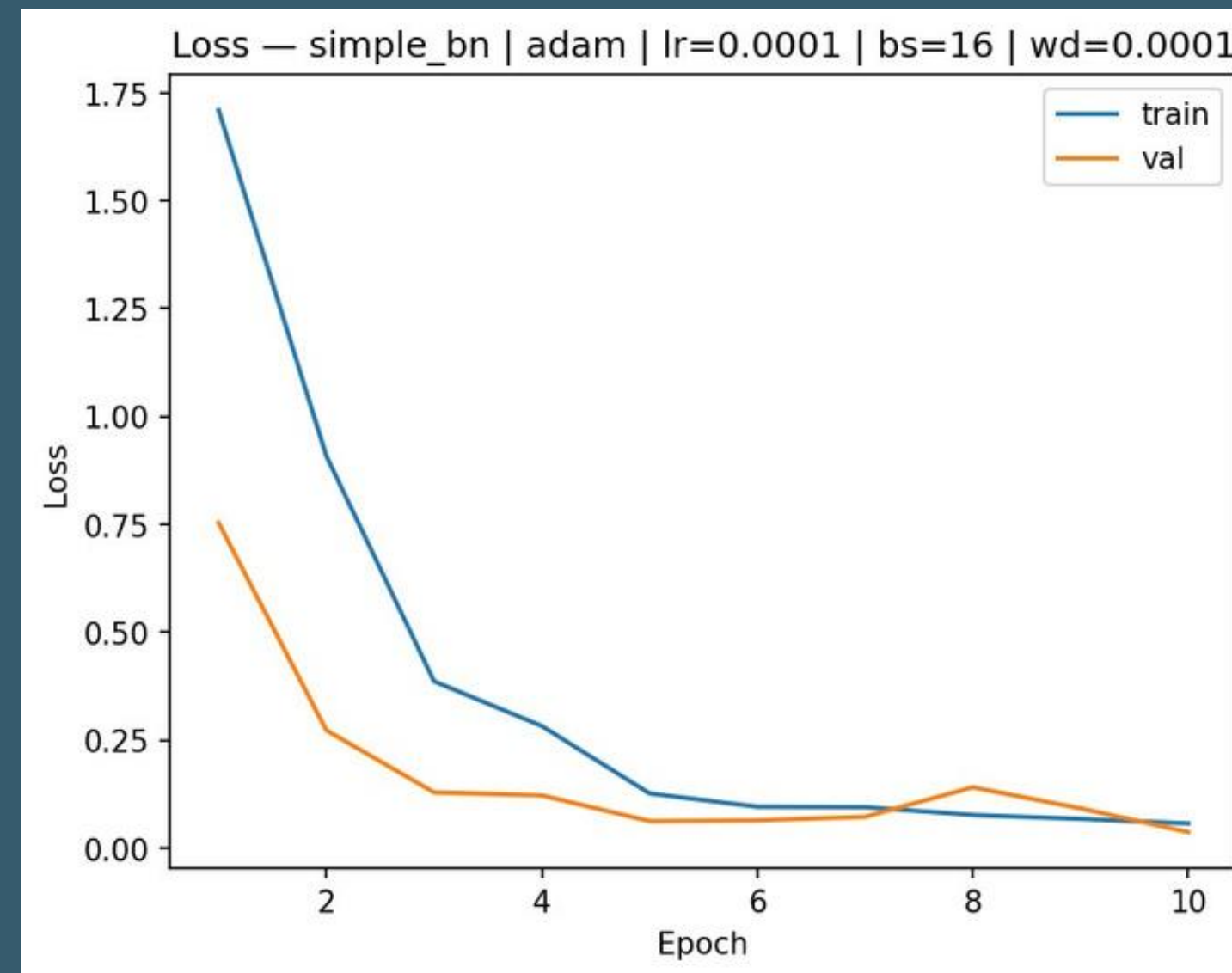
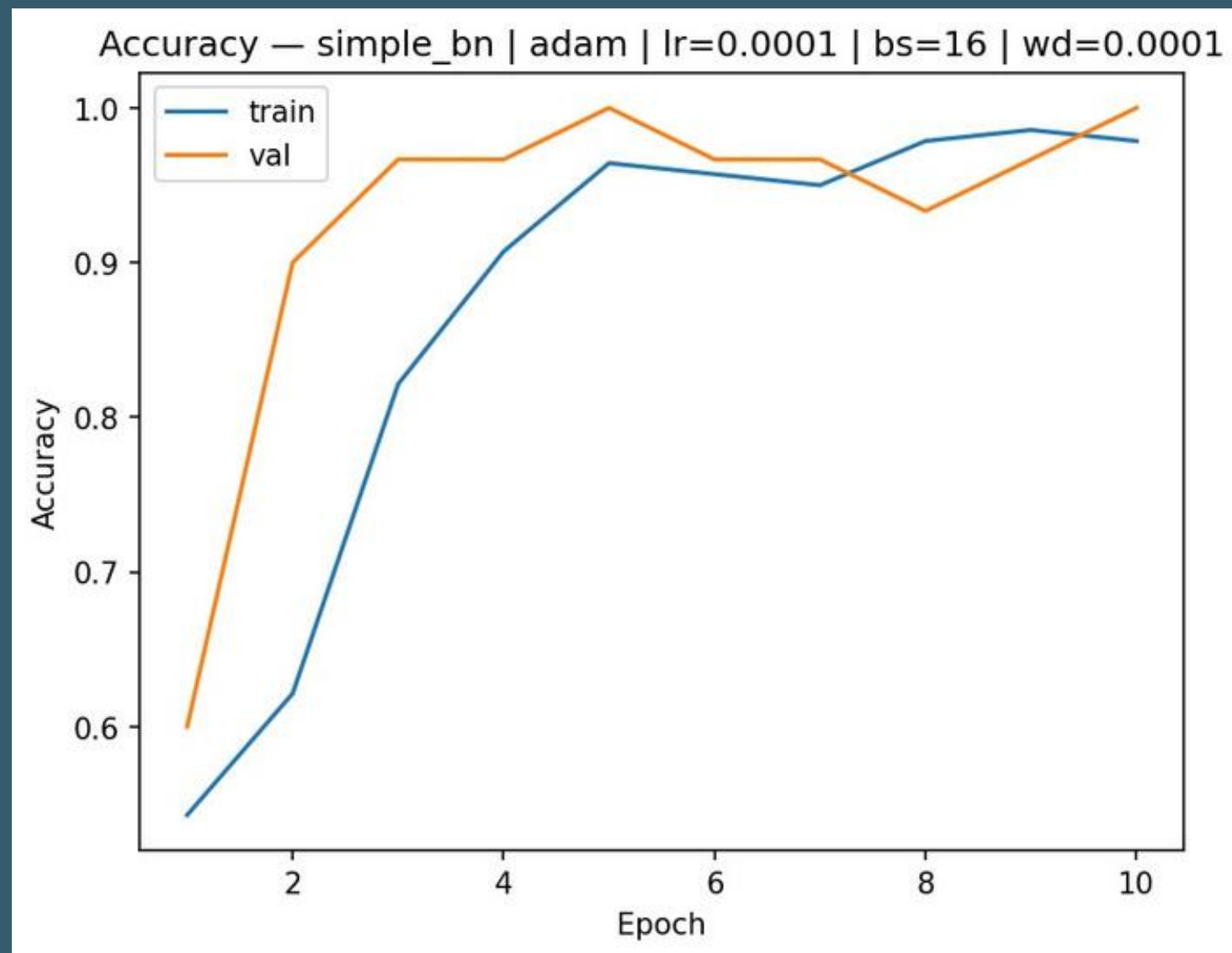
tuning

Step 5 (Fine-tune): reaches $\sim 97\%$ val by epoch 3, best val 100% Test: $\sim 93.3\%$ (similar to from-scratch baseline)

Takeaway: Pretraining on CIFAR-10 noticeably improved convergence speed during fine-tuning. However, the learned features were less aligned with our portrait-based binary task compared to ImageNet features, which later transferred more effectively.

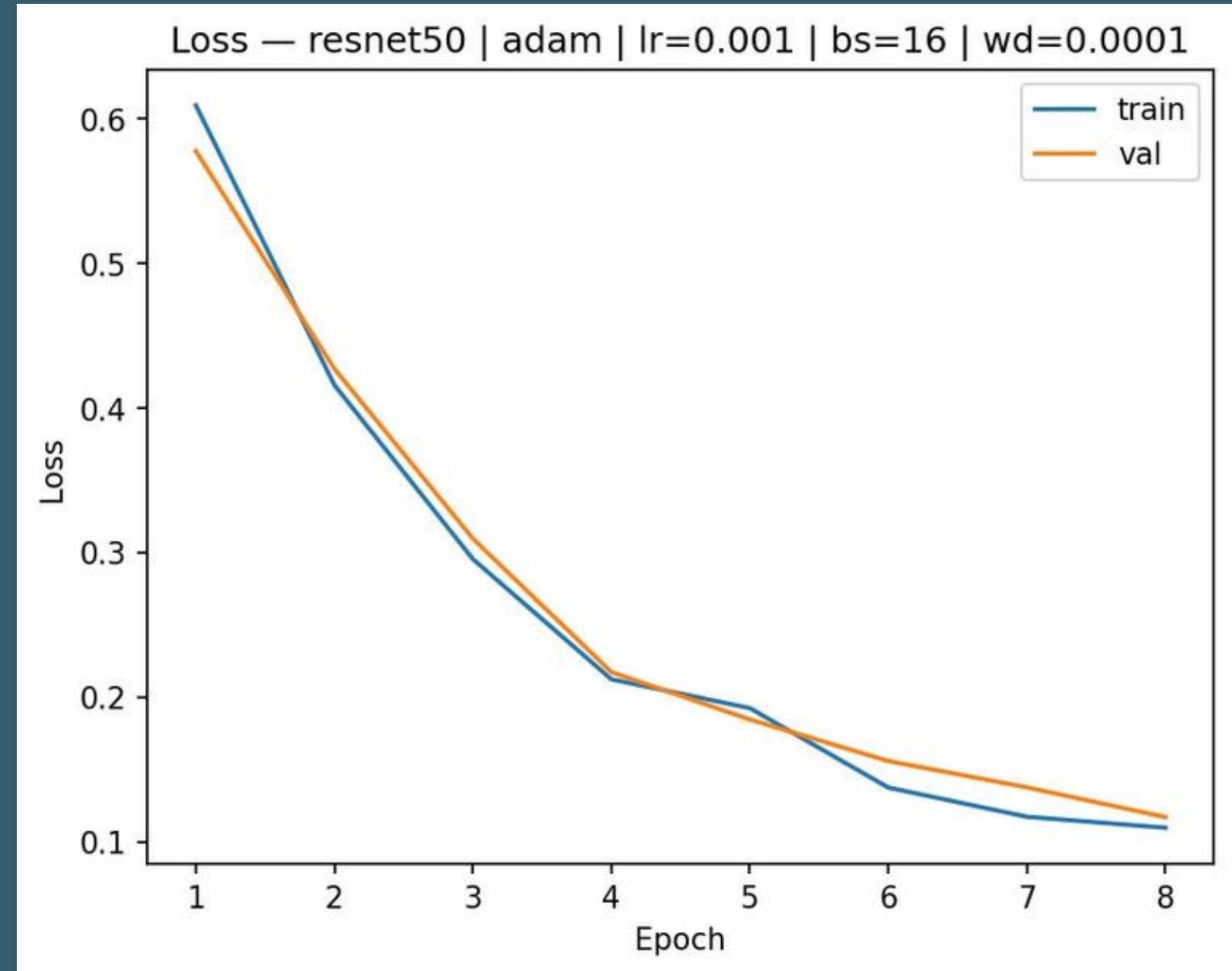
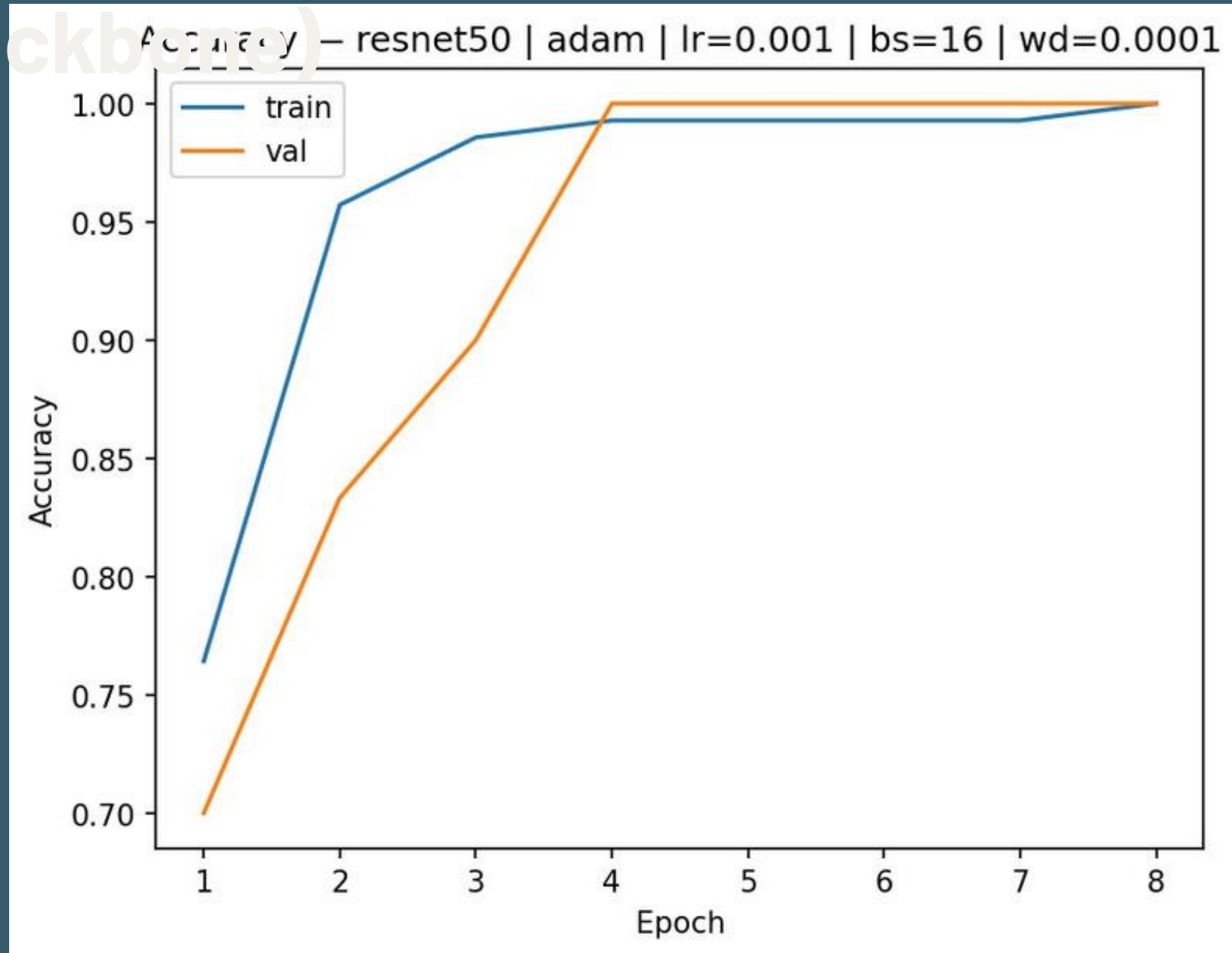
Implication: CIFAR pretraining helps optimization, but does not necessarily improve final performance when the source domain differs.

Figure: Training pipeline – pretrain on CIFAR-10 $\rightarrow$ fine-tune on the portrait dataset (binary classification).



Step 6: ResNet50 Transfer Learning (Frozen

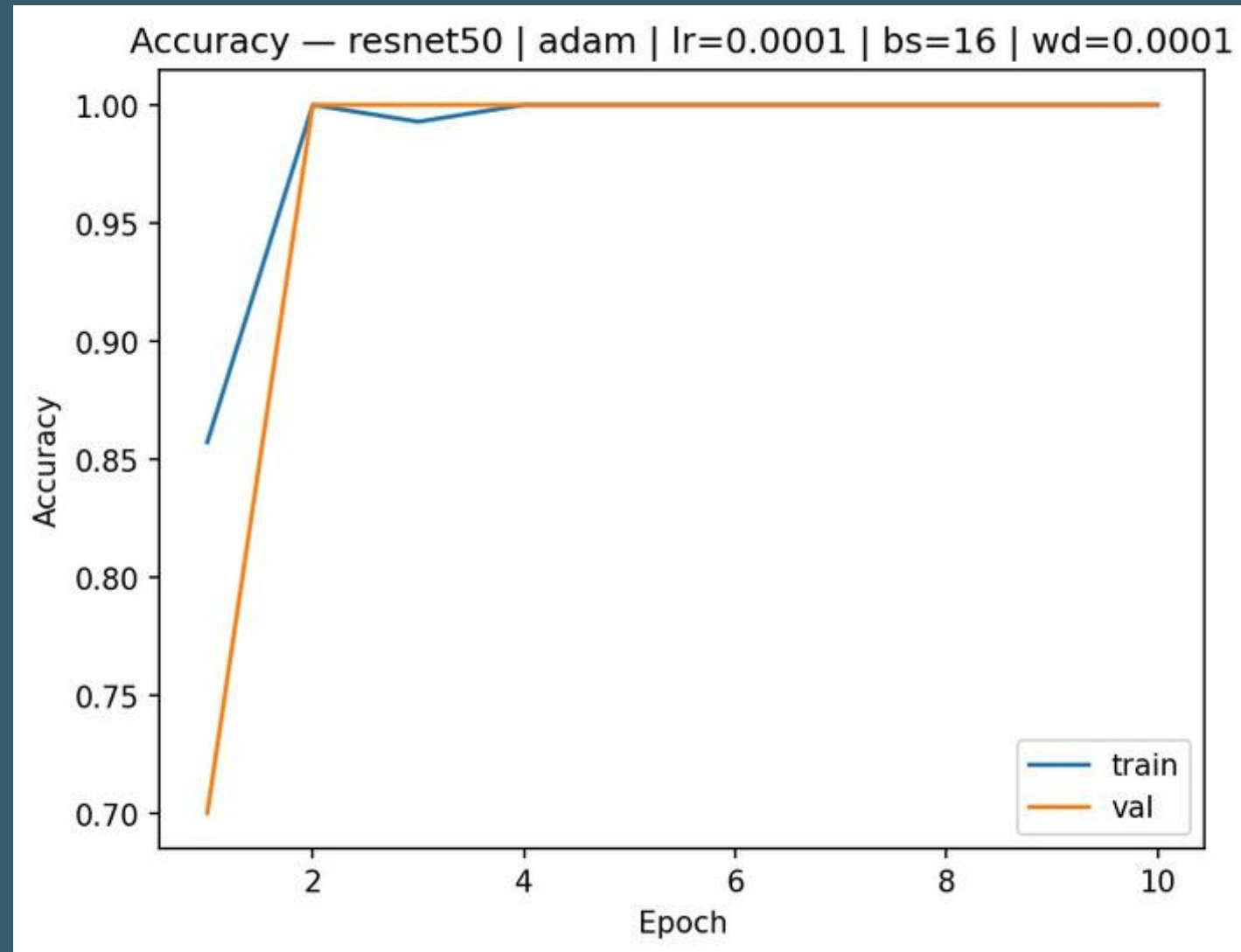
Backbone)



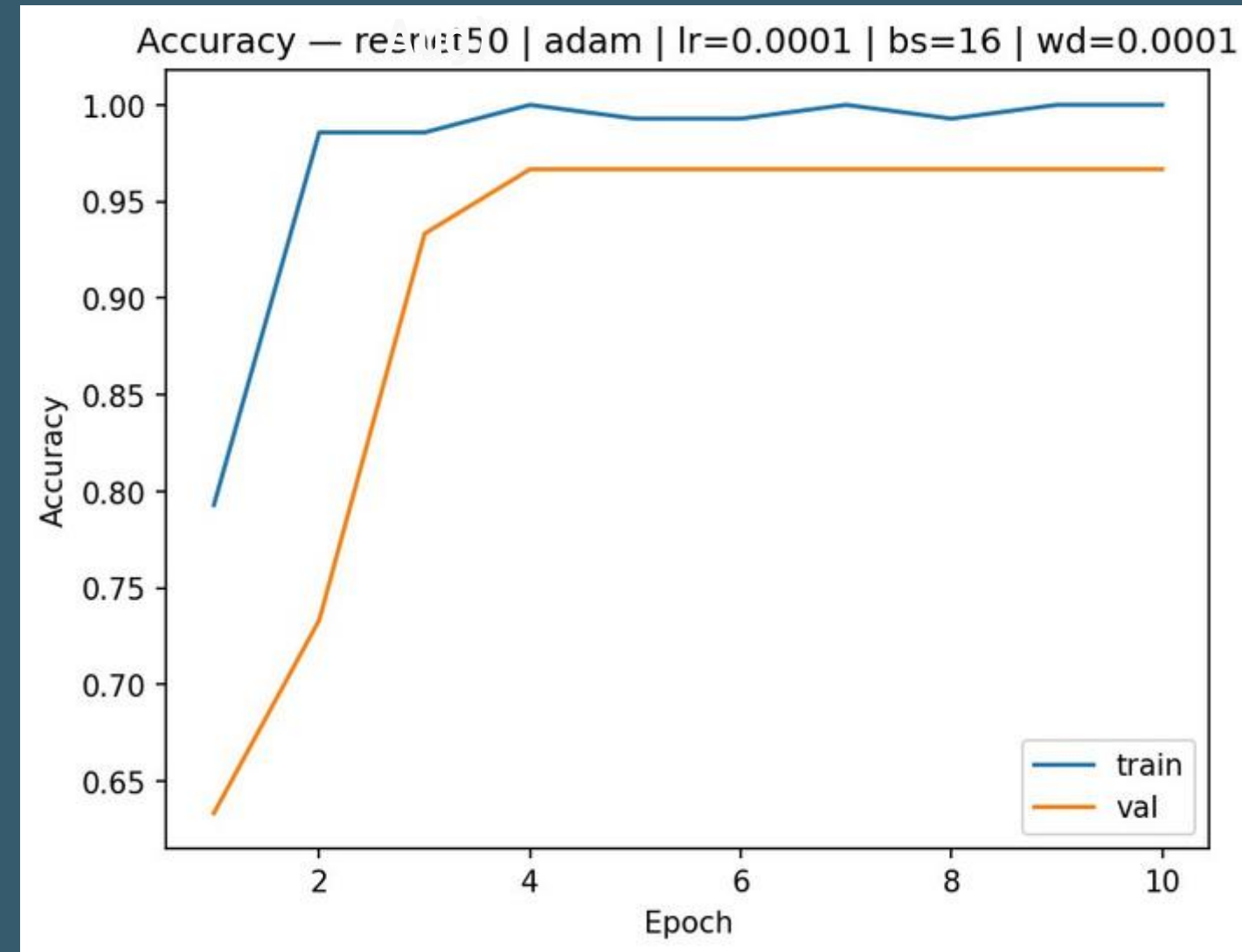
- Setup: ImageNet-pretrained ResNet50, frozen backbone (train classifier only), ImageNet normalization small dataset → reduce overfitting risk + faster training
- Training dynamics: very fast + stable; val=100% by ~epoch 4
- Test: Acc=100% (strong transferable features)

Step 7- ResNet50 Fine-tuning (Augmentation Ablation)

Fine-tune (No Aug)



Fine-tune (Strong)



Result: Test 100% in both settings

Effect: strong augmentation slows optimization and reduces peak val (~96.7%)

Interpretation: Performance saturates due to limited dataset complexity/size

Step	Model	Key change / setting	Best Val Acc	Test Acc	Notes
1	SimpleCNN	From scratch (Adam, lr=1e-3)	~100%	93.30%	Fast convergence validation spikes (v)
2	SimpleCNN + BN	+BatchNorm (Dropout p=0.2 kept)	~100%	100%	Fastest convergence optimization on
3a	SimpleCNN + BN	Dropout ablation (p = 0 / 0.2 / 0.5)	~100%	100%	p=0.2 most stable; spikes; p=0.5 sl
3b	SimpleCNN + BN	Weight decay (wd = 0 / 1e-4 / 1e-3)	~100%	100%	wd=1e-4 most stab shows spike
3c	SimpleCNN + BN	Strong augmentation	~96–100%	96.70%	Harder optimizati variance; robust
4	SimpleCNN + BN	Pretrain on CIFAR-10	N/A	N/A	CIFAR accuracy ~28 (pretraining st
5	SimpleCNN + BN	Fine-tune from CIFAR	100%	93.30%	Faster convergence by epoch 3
6	ResNet50	ImageNet pretrained, frozen backbone	100%	100%	Very fast + stable transfer featu
7	ResNet50	Fine-tune (no aug / strong aug)	100% / ~96.7%	100%	Aug slows converg generalization

Overall Synthesis: The adoption of Batch Normalization led to the most significant improvement in model stability and convergence speed. Transfer learning using Res Net 50 pre-trained on Image Net yielded the highest overall performance. The Adam algorithm proved reliable and consistent across all experiments performed.

Final Takeaways – Key Findings

Best-performing approach: ResNet50 (ImageNet) transfer learning (frozen + fine-tuning) achieved 100% test accuracy with the fastest and most stable convergence.

Most impactful architectural change (from-scratch CNNs): Batch Normalization – accelerated learning and reduced validation instability.

Regularization insight: Dropout ($p=0.2$) and weight decay ($1e-4$) mainly improved training stability, while final accuracy saturated due to task simplicity.

Augmentation insight: Strong augmentation increased difficulty and robustness, but did not improve final results for this dataset (slower convergence, lower validation).

Overall conclusion: Transfer learning provided the largest performance jump and the most reliable generalization in this project.

Final model: ResNet50 + ImageNet normalization (best run) | Test Acc: 100%

Note: test set size is 30 images (fixed split, seed=42); perfect accuracy may be sensitive to the split

Reflection & Future Work

Reflection (Personal) : We enjoyed working on this project and learned that strong results come from balancing multiple factors – not only model design, but also:

- data quality and consistency (preprocessing, normalization)
- evaluation methodology (fixed splits, reproducibility)
- choosing the right strategy (transfer learning vs. training from scratch)

We also learned that in practice, improving the dataset itself (more data, cleaner labels, more diversity) can matter more than increasing model complexity.

Main Challenge: Small dataset (N=200) → performance saturates quickly; limited ability to differentiate methods reliably.

Future Work:

- Expand dataset size and diversity (more identities, lighting, backgrounds).
- Use cross-validation instead of a single split.
- Add deeper analysis: confidence distributions, error inspection, embedding visualization (t-SNE).